

CSE 208: Data Structures and Algorithms II

Sessional

Performance Analysis of Hashing Methods

Humaira Ali Shuvra
Roll: 2205126

1. Hash Functions Used

We used two hash functions for this experiment.

The **first hash function** `hash1` is a polynomial rolling hash function. This takes a string s as an input and finds its hash value.

$$\text{hash1}(s) = \left(\left(\sum_{i=0}^{\text{len}} s_i^i \right) \bmod M \right) \bmod N$$

Here, s_i is the ASCII value of the i th character of the string and len is the length of the string.

The **second hash function** `hash2` is the *FNV1-a* hash function. Here the value of the hash is initialised as 2166136261 and then at each step, XORed with characters of s and multiplied by 16777619. The Final output is given after modulus with N to fit in our table size.

2. Constants Used

- Table size N was initialised as 8000 but then rounded up to the closest prime number.
- The prime M for `hash1` was chosen as $10^9 + 7$.
- The constants chosen for the `hash2` functions are the ones well known for good distribution.

3. Results

We averaged multiple trials to get the following data:

For load factor 0.4

Method	Hash1 Function					Hash2 Function				
	# of Collisions during insertion	Before Deletion		After Deletion		# of Collisions during insertion	Before Deletion		After Deletion	
		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes
Separate Chaining with balanced BST	629	84.375	N/A	121.875	N/A	662	68.75	N/A	90.625	N/A
Linear Probing with Step Adjustment	981	131.25	1.18125	137.5	2.3875	1098	65.625	1.32187	90.625	1.32187
Double Hashing	870	93.75	1.19687	146.875	2.225	866	87.5	1.28125	115.625	1.30625

Figure 1: Performance Metrics for Load Factor 0.4

For load factor 0.5

Method	Hash1 Function					Hash2 Function				
	# of Collisions during insertion	Before Deletion		After Deletion		# of Collisions during insertion	Before Deletion		After Deletion	
		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes
Separate Chaining with balanced BST	978	95	N/A	220	N/A	1003	67.5	N/A	160	N/A
Linear Probing with Step Adjustment	2109	100	1.55	270	3.585	1963	70	1.54	162.5	1.4575
Double Hashing	1507	107.5	1.3925	272.5	2.47	1615	90	1.4075	207.5	1.4425

Figure 2: Performance Metrics for Load Factor 0.5

For load factor 0.6

Method	Hash1 Function					Hash2 Function				
	# of Collisions during insertion	Before Deletion		After Deletion		# of Collisions during insertion	Before Deletion		After Deletion	
		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes
Separate Chaining with balanced BST	1440	87.5	N/A	300	N/A	1418	62.5	N/A	216.667	N/A
Linear Probing with Step Adjustment	3532	95.8333	1.65417	372.917	4.2125	3580	64.5833	1.58333	220.833	1.77292
Double Hashing	2550	95.8333	1.45833	366.667	2.84792	2471	83.3333	1.40417	285.417	1.70833

Figure 3: Performance Metrics for Load Factor 0.6

For load factor 0.7

Method	Hash1 Function					Hash2 Function				
	# of Collisions during insertion	Before Deletion		After Deletion		# of Collisions during insertion	Before Deletion		After Deletion	
		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes
Separate Chaining with balanced BST	1982	78.5714	N/A	362.5	N/A	2000	55.3571	N/A	258.929	N/A
Linear Probing with Step Adjustment	7263	96.4286	2.31786	460.714	7.05	6583	66.0714	2.03214	271.429	1.9875
Double Hashing	4048	96.4286	1.76607	444.643	3.55179	4038	82.1429	1.64107	348.214	1.83393

Figure 4: Performance Metrics for Load Factor 0.7

For load factor 0.8

Method	Hash1 Function					Hash2 Function				
	# of Collisions during insertion	Before Deletion		After Deletion		# of Collisions during insertion	Before Deletion		After Deletion	
		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes
Separate Chaining with balanced BST	2508	79.6875	N/A	415.625	N/A	2490	53.125	N/A	298.438	N/A
Linear Probing with Step Adjustment	13744	96.875	3.31875	557.812	12.2203	11747	65.625	2.66719	325	2.85469
Double Hashing	6304	95.3125	1.99219	517.188	4.48125	6721	79.6875	1.88437	409.375	2.40781

Figure 5: Performance Metrics for Load Factor 0.8

For load factor 0.9

Method	Hash1 Function					Hash2 Function				
	# of Collisions during insertion	Before Deletion		After Deletion		# of Collisions during insertion	Before Deletion		After Deletion	
		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes		Avg Search Time	Avg Probes	Avg Search Time	Avg Probes
Separate Chaining with balanced BST	3214	70.8333	N/A	465.278	N/A	3335	50	N/A	330.556	N/A
Linear Probing with Step Adjustment	33968	104.167	5.4375	716.667	34.6514	35577	83.3333	7.74444	386.111	27.06806
Double Hashing	11188	90.2778	2.45694	590.278	7.8125	11594	83.3333	2.59306	463.889	3.61389

Figure 6: Performance Metrics for Load Factor 0.9

4. Discussion

The load factor α has a significant impact on the performance of all hashing methods in terms of collision count, average probe count, and search time. As α increases from 0.4 to 0.9, all methods experience a deterioration in performance, but the extent varies by technique and hash function.

Linear Probing: Linear probing is the most sensitive to increases in load factor. At low load (e.g., $\alpha = 0.4$), collision counts and probe counts are relatively low, but as α increases, these metrics increase dramatically. At $\alpha = 0.9$, the collision count exceeds 33,000, and the average probe count after deletion rises to over 34 for **hash1**. This sharp rise is due to primary clustering, where long contiguous sequences of occupied slots develop. This

also significantly increases search time, from 130ns to over 700ns after deletion.

Double Hashing: Double hashing handles higher load factors better than linear probing. Although the collision and probe counts increase with α , the growth is more gradual. For example, the average probe count after deletion for `hash2` at $\alpha = 0.9$ is only around 3.6, compared to over 27 for linear probing. This is because double hashing reduces clustering by using a second hash function to determine the probe step size, leading to more uniform probing.

Chaining: Chaining performs the most consistently across different load factors. Even at $\alpha = 0.9$, the collision counts are around 3200, and search times remain under 500ns. Since chaining allows multiple elements to exist at the same index via linked lists, insertion and search remain effective even when the table becomes dense. The collision count increases linearly with α , but does not significantly degrade performance.

Hash Function Comparison: `hash1` and `hash2` show different performance depending on context. `hash2` generally produces fewer probes and faster search times in chaining and double hashing, likely due to better distribution properties of the FNV1-a function. However, in some cases like linear probing, `hash1` performs better under certain load factors, suggesting that hash function effectiveness may vary with collision resolution strategy.

Summary:

- Higher load factors lead to increased collisions, probe counts, and search times in all methods.
- Linear probing suffers heavily from clustering.
- Double hashing is a more robust open addressing strategy.
- Chaining is the least affected by load factor growth.
- Hash function choice can impact performance, with `hash2` generally offering better distribution.